

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Anul 2022 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Definitivat Model 2022	2
SUBIECTUL I (60 de puncte)	2
1. Algoritmul de sortare prin metoda Bulelor (Bubble Sort)	2
a) Descriere în limbaj natural și exemplificare pe 7 numere	2
b) Complexitatea algoritmului	2
c) Problemă completă (Bubble Sort)	2
2. Organizarea logică a datelor în cadrul unui sistem de operare	3
3. Programare: Cifre din mulțimea {0, 1, 2}	3
4. Baze de date: Companie Aviatică	5
II. Rezolvare Definitivat Varianta 3 (20 iulie 2022)	6
SUBIECTUL I (60 de puncte)	6
1. Algoritmul de sortare prin Inserție (Insertion Sort)	6
a) Descriere în limbaj natural și exemplificare pe 7 numere	6
b) Complexitatea algoritmului	7
c) Problemă completă (Sortare prin inserție)	7
2. Rețele: Aplicații colaborative	8
3. Programare: Permutare circulară cifre	8
4. Baze de date: Curse de cai	10
SUBIECTUL al II-lea - Completare pentru Model 2022 (30 de puncte)	11
1. Strategie didactică pentru grafuri neorientate	11
2. Itemi cu alegere multiplă	11
SUBIECTUL al II-lea - Completare pentru Varianta 3 2022 (30 de puncte)	11
1. Strategie didactică pentru structuri de date	11
2. Itemi de completare	11

I. Rezolvare Definitivat Model 2022

SUBIECTUL I (60 de puncte)

1. Algoritmul de sortare prin metoda Bulelor (Bubble Sort)

a) Descriere în limbaj natural și exemplificare pe 7 numere

- **Descriere:** Algoritmul parcurge tabloul în mod repetat și compară elementele adiacente. Dacă două elemente consecutive nu sunt în ordinea corectă (crescătoare), acestea sunt interschimbate. Parcurgerile se repetă până când nu mai are loc nicio interschimbare într-o trecere completă.
- **Exemplificare pentru tabloul:** [25, 14, 30, 8, 12, 5, 20] ($N = 7$)
 - **Trecerea 1:**
 - $25 > 14 \rightarrow$ swap $\rightarrow$ [14, 25, 30, 8, 12, 5, 20]
 - $25 < 30 \rightarrow$ no swap
 - $30 > 8 \rightarrow$ swap $\rightarrow$ [14, 25, 8, 30, 12, 5, 20]
 - $30 > 12 \rightarrow$ swap $\rightarrow$ [14, 25, 8, 12, 30, 5, 20]
 - $30 > 5 \rightarrow$ swap $\rightarrow$ [14, 25, 8, 12, 5, 30, 20]
 - $30 > 20 \rightarrow$ swap $\rightarrow$ [14, 25, 8, 12, 5, 20, 30] (30 a ajuns pe poziția corectă).
 - **Trecerea 2:** Parcurgem până la penultimul element:
 - Interschimbările succesive aduc elementul 25 pe penultima poziție: [14, 8, 12, 5, 20, 25, 30].
 - **Trecerile 3, 4, 5, 6** continuă în același mod până când vectorul devine complet sortat: [5, 8, 12, 14, 20, 25, 30].

b) Complexitatea algoritmului

- **Cazul cel mai nefavorabil (Worst case):** Vectorul este sortat descrescător. Se efectuează $O(N^2)$ comparații și interschimbări.
- **Cazul cel mai favorabil (Best case):** Vectorul este deja sortat. Cu o optimizare ce folosește un fanion (flag) de verificare a interschimbărilor, algoritmul se oprește după prima trecere: $O(N)$.
- **Cazul mediu (Average case):** Complexitatea timpului este de ordinul $O(N^2)$.

c) Problemă completă (Bubble Sort)

• Cod C++:

```
#include <iostream>
using namespace std;
void bubbleSort(int A[], int n) {
    bool sortat;
    do {
        sortat = true;
        for (int i = 0; i < n - 1; ++i) {
            if (A[i] > A[i + 1]) {
                swap(A[i], A[i + 1]);
                sortat = false;
            }
        }
    } while (!sortat);
}
int main() {
```

```
int A[] = {25, 14, 30, 8, 12, 5, 20};
bubbleSort(A, 7);
for (int x : A) cout << x << " ";
return 0;
}
```

2. Organizarea logică a datelor în cadrul unui sistem de operare

• Structura sistemului de fișiere:

1. **Partiție:** O diviziune logică a unui hard disk fizic tratată de SO ca un volum independent.
2. **Director:** O structură logică de tip container ce conține referințe către fișiere și alte directoare.
3. **Fișier:** O colecție organizată de informații stocată pe un mediu extern.

• Operații:

- **Formatare:** Pregătirea unei partiții pentru a fi folosită de SO prin crearea structurii sistemului de fișiere (ex. NTFS, FAT32).
- **Copiere, Mutare, Redenumire, Ștergere** de directoare/fișiere.

3. Programare: Cifre din mulțimea {0, 1, 2}

- **Descriere:** Subprogramul `cifDistMax` numără cifrele distincte și determină cifra maximă a lui n . Programul principal generează recursiv toate numerele formate exclusiv din cifrele {0, 1, 2}, le ordonează și utilizează căutarea binară pentru a determina cel mai mic număr din intervalul $[x, y]$.

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <queue>
using namespace std;

void cifDistMax(long long n, int &nrd, int &cma) {
    bool ap[10] = {false};
    nrd = 0; cma = 0;
    long long temp = n;
    if (temp == 0) { nrd = 1; cma = 0; return; }
    while (temp > 0) {
        int cif = temp % 10;
        if (!ap[cif]) { ap[cif] = true; nrd++; }
        if (cif > cma) cma = cif;
        temp /= 10;
    }
}

vector<long long> vals;
void genereaza() {
    vals.push_back(0);
    queue<long long> Q;
    Q.push(1); Q.push(2);
    while (!Q.empty()) {
```

```

        long long u = Q.front(); Q.pop();
        if (u > 1000000000LL) continue;
        vals.push_back(u);
        Q.push(u * 10 + 0);
        Q.push(u * 10 + 1);
        Q.push(u * 10 + 2);
    }
    sort(vals.begin(), vals.end());
}

int main() {
    genereaza();
    long long x, y;
    ifstream fin("def2022.in"); // în cazul în care datele vin din fișier
    // dar enunțul model spune: se citesc de la tastatură x și y, se scrie în
    def2022.out
    if (!(cin >> x >> y)) return 0;

    ofstream fout("def2022.out");
    auto it = lower_bound(vals.begin(), vals.end(), x);
    if (it != vals.end() && *it <= y) {
        fout << *it << "\n";
    } else {
        fout << "nu exista\n";
    }
    fout.close();
    return 0;
}

```

Soluție Pascal:

```

program NumereTernare;
var
    x, y: int64;
    vals: array[1..60000] of int64;
    count, i, st, dr, mid, pos: integer;
    fout: text;

procedure gen(val: int64);
begin
    if val > 1000000000 then exit;
    count := count + 1;
    vals[count] := val;
    gen(val * 10 + 0);
    gen(val * 10 + 1);
    gen(val * 10 + 2);
end;

procedure QuickSort(l, r: integer);
var
    i, j: integer;
    pivot, tmp: int64;
begin
    i := l; j := r;
    pivot := vals[(l + r) div 2];
    repeat
        while vals[i] < pivot do i := i + 1;

```

```

while vals[j] > pivot do j := j - 1;
if i <= j then
begin
    tmp := vals[i]; vals[i] := vals[j]; vals[j] := tmp;
    i := i + 1; j := j - 1;
end;
until i > j;
if l < j then QuickSort(l, j);
if i < r then QuickSort(i, r);
end;

begin
    readln(x, y);
    count := 1;
    vals[1] := 0;
    gen(1);
    gen(2);
    QuickSort(1, count);

    st := 1; dr := count; pos := -1;
    while st <= dr do
    begin
        mid := (st + dr) div 2;
        if vals[mid] >= x then
        begin
            pos := mid;
            dr := mid - 1;
        end
        else st := mid + 1;
    end;

    assign(fout, 'def2022.out');
    rewrite(fout);
    if (pos <> -1) and (vals[pos] <= y) then
        writeln(fout, vals[pos])
    else
        writeln(fout, 'nu exista');
    close(fout);
end.

```

4. Baze de date: Companie Aviatică

Model conceptual:

- CLIENT: id_client, nume, prenume, telefon, email.
- ZBOR: id_zbor, aeroport_sursa, aeroport_destinatie, data_ora_plecure, data_ora_sosire.
- BILET: id_bilet, id_client, id_zbor, mod_plata, data_achizitie, pret.

Relații și reguli:

- CLIENT 1:M BILET; un client poate cumpăra mai multe bilete.
- ZBOR 1:M BILET; un zbor poate avea mai multe bilete.
- **1NF**: date atomice, fără liste de zboruri sau plăți într-un câmp.
- **2NF**: attributele biletului depind de id_bilet; nu există dependențe parțiale.
- **3NF**: datele clientului și ale zborului sunt separate de datele biletului.

- data_ora_sosire > data_ora_plecare, pret >= 0, mod_plata are valori controlate (numerar, card, transfer etc.).

Model fizic:

```
CREATE TABLE CLIENT (
    id_client INT PRIMARY KEY AUTO_INCREMENT,
    nume VARCHAR(80) NOT NULL,
    prenume VARCHAR(80) NOT NULL,
    telefon VARCHAR(30),
    email VARCHAR(120)
);

CREATE TABLE ZBOR (
    id_zbor INT PRIMARY KEY AUTO_INCREMENT,
    aeroport_sursa VARCHAR(80) NOT NULL,
    aeroport_destinatie VARCHAR(80) NOT NULL,
    data_ora_plecare DATETIME NOT NULL,
    data_ora_sosire DATETIME NOT NULL
);

CREATE TABLE BILET (
    id_bilet INT PRIMARY KEY AUTO_INCREMENT,
    id_client INT NOT NULL,
    id_zbor INT NOT NULL,
    mod_plata VARCHAR(30) NOT NULL,
    data_achizitie DATE NOT NULL,
    pret DECIMAL(10,2) CHECK (pret >= 0),
    FOREIGN KEY (id_client) REFERENCES CLIENT(id_client),
    FOREIGN KEY (id_zbor) REFERENCES ZBOR(id_zbor)
);
```

SQL pentru clienții distincți care au plătit numerar:

```
SELECT DISTINCT c.prenume, c.nume
FROM CLIENT c
JOIN BILET b ON c.id_client = b.id_client
WHERE b.mod_plata = 'numerar';
```

II. Rezolvare Definitivat Varianta 3 (20 iulie 2022)

SUBIECTUL I (60 de puncte)

1. Algoritmul de sortare prin Inserție (Insertion Sort)

a) Descriere în limbaj natural și exemplificare pe 7 numere

- **Descriere:** Algoritmul pornește de la al doilea element al tabloului și îl consideră pe rând drept „cheie”. Elementul cheie este comparat cu elementele din stânga sa (deja sortate) și deplasat spre stânga până când își găsește poziția corectă (când se întâlnește un element mai mic sau egal cu el).
- **Exemplificare pentru tabloul:** [35, 12, 45, 8, 20, 15, 30] (N = 7)
 1. **Pasul 1** (cheie = 12): 12 < 35, deplasăm 35. Tabloul devine: [12, 35, 45, 8, 20, 15, 30].
 2. **Pasul 2** (cheie = 45): 45 > 35, rămâne pe loc. Tabloul: [12, 35, 45, 8, 20, 15, 30].

3. **Pasul 3** (cheie = 8): $8 < 45, 8 < 35, 8 < 12$. Deplasăm toate elementele. Tabloul: [8, 12, 35, 45, 20, 15, 30].
4. **Pasul 4** (cheie = 20): $20 < 45, 20 < 35, 20 > 12$. Deplasăm 45 și 35. Tabloul: [8, 12, 20, 35, 45, 15, 30].
5. **Pasul 5** (cheie = 15): $15 < 45, 15 < 35, 15 < 20, 15 > 12$. Deplasăm 45, 35, 20. Tabloul: [8, 12, 15, 20, 35, 45, 30].
6. **Pasul 6** (cheie = 30): $30 < 45, 30 < 35, 30 > 20$. Deplasăm 45 și 35. Tabloul final sortat: [8, 12, 15, 20, 30, 35, 45].

b) Complexitatea algoritmului

- **Cazul cel mai nefavorabil (Worst case):** Vectorul este sortat descrescător. Pentru fiecare element de la poziția i facem i comparații. Complexitatea este $O(N^2)$.
- **Cazul cel mai favorabil (Best case):** Vectorul este deja sortat crescător. Facem o singură comparație pentru fiecare cheie. Complexitatea este $O(N)$.
- **Cazul mediu (Average case):** Complexitatea timpului este tot de ordinul $O(N^2)$.

c) Problemă completă (Sortare prin inserție)

- **Enunț:** Să se ordoneze crescător elementele unui vector folosind sortarea prin inserție.
- **Cod C++:**

```
#include <iostream>
using namespace std;
void insertionSort(int A[], int n) {
    for (int i = 1; i < n; ++i) {
        int cheie = A[i];
        int j = i - 1;
        while (j >= 0 && A[j] > cheie) {
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = cheie;
    }
}
int main() {
    int A[] = {35, 12, 45, 8, 20, 15, 30};
    insertionSort(A, 7);
    for (int x : A) cout << x << " ";
    return 0;
}
```

- **Cod Pascal:**

```
program SortareInsertie;
var
    A: array[1..7] of integer = (35, 12, 45, 8, 20, 15, 30);
    i, j, cheie: integer;
begin
    for i := 2 to 7 do
        begin
            cheie := A[i];
            j := i - 1;
            while (j >= 1) and (A[j] > cheie) do
                begin
                    A[j + 1] := A[j];
                    j := j - 1;
                end
        end
    end
```

```

    end;
    A[j + 1] := cheie;
end;
for i := 1 to 7 do write(A[i], ' ');
end.

```

2. Rețele: Aplicații colaborative

- **Aplicații colaborative:** Programe ce permit utilizatorilor aflați în locații diferite să lucreze în timp real la același document sau proiect (ex. Microsoft Teams, Google Docs).
 - **Dispozitive periferice utilizate:** Cameră web, microfon, căști/boxe.
 - **Beneficii:** Reducerea costurilor de deplasare, flexibilitatea programului de lucru.
 - **Dezavantaje:** Dependența critică de conexiunea la internet, riscuri crescute privind securitatea datelor.
-

3. Programare: Permutare circulară cifre

Soluție C++:

```

#include <iostream>
#include <fstream>
using namespace std;

void circular(long long &n) {
    long long temp = n;
    long long p = 1;
    while (temp >= 10) {
        p *= 10;
        temp /= 10;
    }
    int last = n % 10;
    n = last * p + (n / 10);
}

int main() {
    ifstream fin("def2022.in");
    long long x;
    long long min_odd = -1;
    int count_min = 0;
    while (fin >> x) {
        circular(x);
        if (x % 2 != 0) {
            if (min_odd == -1 || x < min_odd) {
                min_odd = x;
                count_min = 1;
            } else if (x == min_odd) {
                count_min++;
            }
        }
    }
    fin.close();

    if (min_odd == -1) cout << "nu exista\n";
    else cout << min_odd << " " << count_min << "\n";
}

```



```
    return 0;
}
```

Soluție Pascal:

```
program PermutareCifre;
var
    fin: text;
    x, min_odd: int64;
    count_min: integer;

procedure circular(var n: int64);
var
    temp, p: int64;
    last: integer;
begin
    temp := n;
    p := 1;
    while temp >= 10 do
    begin
        p := p * 10;
        temp := temp div 10;
    end;
    last := n mod 10;
    n := last * p + (n div 10);
end;

begin
    assign(fin, 'def2022.in');
    reset(fin);
    min_odd := -1;
    count_min := 0;
    while not eof(fin) do
    begin
        read(fin, x);
        circular(x);
        if x mod 2 <> 0 then
        begin
            if (min_odd = -1) or (x < min_odd) then
            begin
                min_odd := x;
                count_min := 1;
            end
            else if x = min_odd then
                count_min := count_min + 1;
            end;
        end;
    end;
    close(fin);

    if min_odd = -1 then writeln('nu exista')
    else writeln(min_odd, ' ', count_min);
end.
```

—

4. Baze de date: Curse de cai

Model conceptual:

- CAL: id_cal, nume, varsta, nume_proprietar, telefon_proprietar, email_proprietar.
- CURSA: id_cursa, tip_cursa, data_ora, status.
- INSCRIERE: id_cal, id_cursa, a_participat, loc_clasare.

Relații și reguli:

- CAL M:N CURSA, realizată prin INSCRIERE.
- **1NF**: fiecare atribut are valori atomice.
- **2NF**: în INSCRIERE, loc_clasare depinde de perechea (id_cal, id_cursa).
- **3NF**: datele proprietarului sunt stocate la cal, nu repetate la fiecare cursă.
- loc_clasare este NULL dacă un cal nu a participat; varsta > 0.

Model fizic:

```
CREATE TABLE CAL (
    id_cal INT PRIMARY KEY AUTO_INCREMENT,
    nume VARCHAR(100) NOT NULL,
    varsta INT CHECK (varsta > 0),
    nume_proprietar VARCHAR(120),
    telefon_proprietar VARCHAR(30),
    email_proprietar VARCHAR(120)
);
```

```
CREATE TABLE CURSA (
    id_cursa INT PRIMARY KEY AUTO_INCREMENT,
    tip_cursa VARCHAR(50) NOT NULL,
    data_ora DATETIME NOT NULL,
    status VARCHAR(20) NOT NULL
);
```

```
CREATE TABLE INSCRIERE (
    id_cal INT,
    id_cursa INT,
    a_participat BOOLEAN DEFAULT FALSE,
    loc_clasare INT NULL,
    PRIMARY KEY (id_cal, id_cursa),
    FOREIGN KEY (id_cal) REFERENCES CAL(id_cal),
    FOREIGN KEY (id_cursa) REFERENCES CURSA(id_cursa)
);
```

SQL pentru ștergerea cailor neînscriși în ultimii doi ani:

```
DELETE FROM CAL
WHERE id_cal NOT IN (
    SELECT DISTINCT i.id_cal
    FROM INSCRIERE i
    JOIN CURSA c ON i.id_cursa = c.id_cursa
    WHERE c.data_ora >= DATE_SUB(CURDATE(), INTERVAL 2 YEAR)
);
```

—

SUBIECTUL al II-lea - Completare pentru Model 2022 (30 de puncte)

1. Strategie didactică pentru grafuri neorientate

Mijloc: aplicație de reprezentare a grafurilor. **Caracteristică:** vizualizează noduri, muchii și lanțuri. **Avantaj:** sprijină transpunerea unei probleme din limbaj natural în model de graf.

Metodă: învățarea prin descoperire. **Formă:** grupe mici. **Activitate:** construirea grafului unei rețele de drumuri și identificarea componentelor conexe.

Scenariu: Profesorul prezintă o hartă simplificată, elevii transformă localitățile în noduri și drumurile în muchii, apoi identifică lanțuri și subgrafuri.

2. Itemi cu alegere multiplă

Caracteristici: enunț clar, variante plauzibile, un singur răspuns corect. **Reguli:** distractori omogeni, fără răspunsuri de tip „toate”, fără ambiguitate.

- **Item A:** O muchie într-un graf neorientat leagă: A. două noduri B. două fișiere C. două variabile D. două tabele SQL. **Răspuns:** A.
- **Item B:** Care dispozitiv este de intrare? A. Monitor B. Tastatură C. Imprimantă D. Videoproiector. **Răspuns:** B.

SUBIECTUL al II-lea - Completare pentru Varianta 3 2022 (30 de puncte)

1. Strategie didactică pentru structuri de date

Metodă: exercițiul dirijat. **Avantaje:** dezvoltă deprinderi de parcurgere/inserare/ștergere; permite corectarea rapidă a erorilor de pointeri sau indici.

Mijloc: tablă interactivă și mediu de programare. **Formă:** frontal, apoi individual. **Activitate:** inserarea unui element într-o listă simplu înlănțuită.

Scenariu: Profesorul desenează nodurile și legăturile, elevii indică ordinea corectă a actualizării pointerilor, apoi implementează procedura.

2. Itemi de completare

Caracteristici și reguli: răspuns scurt, obiectiv, cerință clară; se evită formulările cu mai multe completări posibile.

- **Item A:** Într-o listă simplu înlănțuită, fiecare nod conține informația utilă și adresa nodului [spațiu liber]. **Răspuns:** următor.
- **Item B:** Memoria care își pierde conținutul la oprirea calculatorului se numește [spațiu liber]. **Răspuns:** RAM / memorie volatilă.